Reverse- and Reengineering
Homework 3

Winter 2026/27 — Prof. Dr. Christian Hammer
Date: 3.11.2026 — Dr. Mohammad Rezaalipour

This sheet is not for submission. However, we strongly encourage you to solve the problems and bring your solutions to the tutorials for discussion.

1 x86 Assembly

The two programs below are assembly language programs for two functions, `f1` and `f2`. Write the equivalent C programs for both.

Listing 1: `f1` function

```
1 push   rbp
2 mov    rbp, rsp
3 mov    dword ptr [rbp - 4], 15
4 mov    dword ptr [rbp - 8], 2
5 mov    eax, dword ptr [rbp - 4]
6 mov    ecx, dword ptr [rbp - 8]
7 sar    eax, cl
8 pop    rbp
9 ret
```

Listing 2: `f2` function

```
1 push   rbp
2 mov    rbp, rsp
3 mov    dword ptr [rbp - 4], 10
4 mov    dword ptr [rbp - 8], 2
5 mov    eax, dword ptr [rbp - 4]
6 mov    ecx, dword ptr [rbp - 8]
7 shl    eax, cl
8 pop    rbp
9 ret
```

2 Control Flow Graphs (CFG)

Consider the x86 assembly program below containing the functions `max` and `main`.

1. Guess what the function `max` computes.
2. Draw both the intra-procedural and inter-procedural CFGs for this example.

```
max:
        push    rbp
        mov     rbp, rsp
        sub     rsp, 16
        mov     dword ptr [rbp - 8], edi
        mov     dword ptr [rbp - 12], esi
        mov     dword ptr [rbp - 16], edx
        mov     eax, dword ptr [rbp - 8]
        cmp     eax, dword ptr [rbp - 12]
        jl      .L3
        mov     eax, dword ptr [rbp - 8]
        cmp     eax, dword ptr [rbp - 16]
        jl      .L3
        mov     esi, dword ptr [rbp - 8]
        lea     rdi, [rip + .L.str]
        mov     al, 0
        call    printf
.L3:
        mov     eax, dword ptr [rbp - 12]
        cmp     eax, dword ptr [rbp - 8]
        jl      .L6
        mov     eax, dword ptr [rbp - 12]
        cmp     eax, dword ptr [rbp - 16]
        jl      .L6
        mov     esi, dword ptr [rbp - 12]
        lea     rdi, [rip + .L.str]
        mov     al, 0
        call    printf
.L6:
        mov     eax, dword ptr [rbp - 16]
        cmp     eax, dword ptr [rbp - 8]
        jl      .L9
        mov     eax, dword ptr [rbp - 16]
        cmp     eax, dword ptr [rbp - 12]
        jl      .L9
        mov     esi, dword ptr [rbp - 16]
        lea     rdi, [rip + .L.str]
        mov     al, 0
        call    printf
.L9:
        mov     eax, dword ptr [rbp - 4]
        add     rsp, 16
        pop     rbp
        ret
main:
        push    rbp
        mov     rbp, rsp
        sub     rsp, 16
        mov     dword ptr [rbp - 4], 0
        mov     dword ptr [rbp - 8], 100
        mov     dword ptr [rbp - 12], 200
        mov     dword ptr [rbp - 16], 400
        mov     edi, dword ptr [rbp - 8]
        mov     esi, dword ptr [rbp - 12]
        mov     edx, dword ptr [rbp - 16]
        call    max
        xor     eax, eax
        add     rsp, 16
        pop     rbp
        ret
.L.str:
        .asciz "%d is the largest number."
```